

DAA - Design and Analysis of Algorithms

Session 1 Lab Experiments

Name: J O Joanna Divine

Reg No: 192572003

Question 1: First Palindromic String in an Array

Problem: Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "".

Program (C):

```
#include <stdio.h>
#include <string.h>

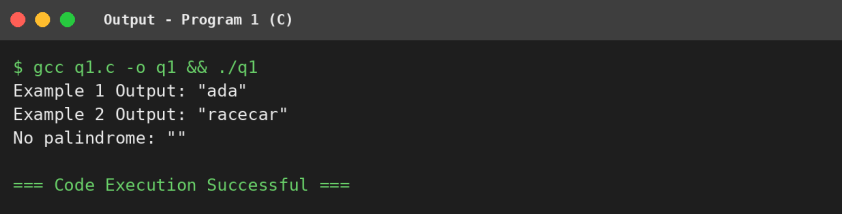
int isPalindrome(char s[]) {
    int i = 0, j = strlen(s) - 1;
    while (i < j) {
        if (s[i] != s[j])
            return 0;
        i++;
        j--;
    }
    return 1;
}

char *firstPalindrome(char words[][20], int n) {
    int i;
    for (i = 0; i < n; i++) {
        if (isPalindrome(words[i]))
            return words[i];
    }
    return "";
}

int main() {
    char w1[5][20] = {"abc", "car", "ada", "racecar", "cool"};
    char w2[2][20] = {"notapalindrome", "racecar"};
    char w3[2][20] = {"abc", "def"};

    printf("Example 1 Output: \"%s\\n\"", firstPalindrome(w1, 5));
    printf("Example 2 Output: \"%s\\n\"", firstPalindrome(w2, 2));
    printf("No palindrome: \"%s\\n\"", firstPalindrome(w3, 2));
    return 0;
}
```

Output:



```
Output - Program 1 (C)

$ gcc q1.c -o q1 && ./q1
Example 1 Output: "ada"
Example 2 Output: "racecar"
No palindrome: ""

=== Code Execution Successful ===
```

Complexity: Time: $O(N \times L)$ where L is the average word length. Space: $O(1)$.

Question 2: Count Common Elements Between Two Arrays

Problem: Given integer arrays `nums1` and `nums2` of sizes `n` and `m`, return `[answer1, answer2]`, where `answer1` is the number of indices `i` such that `nums1[i]` exists in `nums2`, and `answer2` is the number of indices `i` such that `nums2[i]` exists in `nums1`.

Program (C):

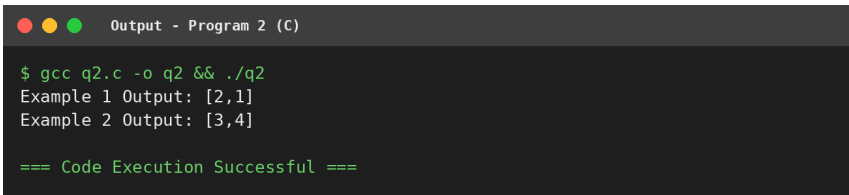
```
#include <stdio.h>

int exists(int arr[], int size, int x) {
    int i;
    for (i = 0; i < size; i++) {
        if (arr[i] == x)
            return 1;
    }
    return 0;
}

void solve(int nums1[], int n, int nums2[], int m, int ex) {
    int i, ans1 = 0, ans2 = 0;
    for (i = 0; i < n; i++)
        if (exists(nums2, m, nums1[i]))
            ans1++;
    for (i = 0; i < m; i++)
        if (exists(nums1, n, nums2[i]))
            ans2++;
    printf("Example %d Output: [%d,%d]\n", ex, ans1, ans2);
}

int main() {
    int a1[] = {2, 3, 2};
    int b1[] = {1, 2};
    int a2[] = {4, 3, 2, 3, 1};
    int b2[] = {2, 2, 5, 2, 3, 6};
    solve(a1, 3, b1, 2, 1);
    solve(a2, 5, b2, 6, 2);
    return 0;
}
```

Output:



```
Output - Program 2 (C)

$ gcc q2.c -o q2 && ./q2
Example 1 Output: [2,1]
Example 2 Output: [3,4]

=== Code Execution Successful ===
```

Complexity: Time: $O(n \times m)$. Space: $O(1)$.

Question 3: Sum of Squares of Distinct Counts of All Subarrays

Problem: Given a 0-indexed integer array `nums`, return the sum of the squares of the distinct counts of all its subarrays.

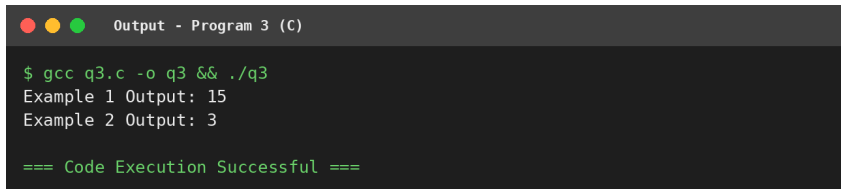
Program (C):

```
#include <stdio.h>
#include <string.h>

int sumOfSquares(int nums[], int n) {
    int i, j, total = 0;
    for (i = 0; i < n; i++) {
        int seen[101];
        int distinct = 0;
        memset(seen, 0, sizeof(seen));
        for (j = i; j < n; j++) {
            if (seen[nums[j]] == 0) {
                seen[nums[j]] = 1;
                distinct++;
            }
            total = total + distinct * distinct;
        }
    }
    return total;
}

int main() {
    int a[] = {1, 2, 1};
    int b[] = {1, 1};
    printf("Example 1 Output: %d\n", sumOfSquares(a, 3));
    printf("Example 2 Output: %d\n", sumOfSquares(b, 2));
    return 0;
}
```

Output:



```
Output - Program 3 (C)

$ gcc q3.c -o q3 && ./q3
Example 1 Output: 15
Example 2 Output: 3

=== Code Execution Successful ===
```

Complexity: Time: $O(n^2)$. Space: $O(1)$ (fixed-size table for values 1..100).

Question 4: Count Equal Pairs Whose Index Product is Divisible by k

Problem: Given a 0-indexed integer array `nums` of length `n` and an integer `k`, return the number of pairs `(i, j)` where $0 \leq i < j < n$, `nums[i] == nums[j]` and $(i * j)$ is divisible by `k`.

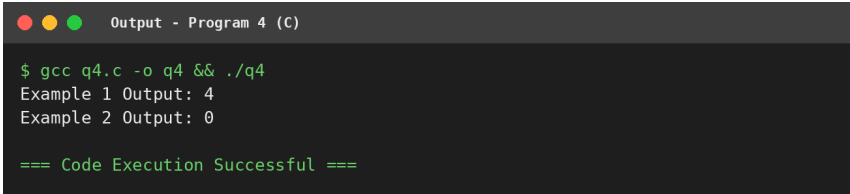
Program (C):

```
#include <stdio.h>

int countPairs(int nums[], int n, int k) {
    int i, j, count = 0;
    for (i = 0; i < n; i++) {
        for (j = i + 1; j < n; j++) {
            if (nums[i] == nums[j] && (i * j) % k == 0)
                count++;
        }
    }
    return count;
}

int main() {
    int a[] = {3, 1, 2, 2, 2, 1, 3};
    int b[] = {1, 2, 3, 4};
    printf("Example 1 Output: %d\n", countPairs(a, 7, 2));
    printf("Example 2 Output: %d\n", countPairs(b, 4, 1));
    return 0;
}
```

Output:



```
$ gcc q4.c -o q4 && ./q4
Example 1 Output: 4
Example 2 Output: 0

=== Code Execution Successful ===
```

Complexity: Time: $O(n^2)$. Space: $O(1)$.

Question 5: Program for the Given Test Cases with Least Time Complexity

Problem: Test cases: {1,2,3,4,5} -> 5, {7,7,7,7,7} -> 7, {-10,2,3,-4,5} -> 5. The expected outputs show the task is to find the maximum element.

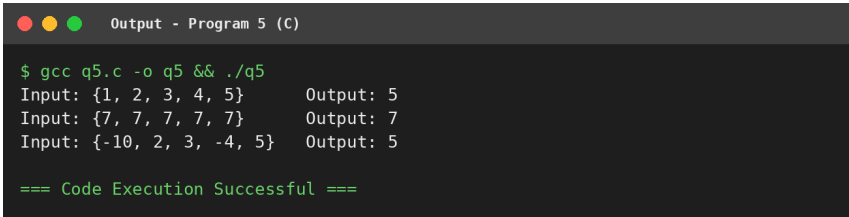
Program (C):

```
#include <stdio.h>

/* one pass through the array - time complexity O(n) */
int findMax(int arr[], int n) {
    int i, max = arr[0];
    for (i = 1; i < n; i++) {
        if (arr[i] > max)
            max = arr[i];
    }
    return max;
}

int main() {
    int t1[] = {1, 2, 3, 4, 5};
    int t2[] = {7, 7, 7, 7, 7};
    int t3[] = {-10, 2, 3, -4, 5};
    printf("Input: {1, 2, 3, 4, 5}    Output: %d\n", findMax(t1, 5));
    printf("Input: {7, 7, 7, 7, 7}    Output: %d\n", findMax(t2, 5));
    printf("Input: {-10, 2, 3, -4, 5}  Output: %d\n", findMax(t3, 5));
    return 0;
}
```

Output:



```
Output - Program 5 (C)

$ gcc q5.c -o q5 && ./q5
Input: {1, 2, 3, 4, 5}    Output: 5
Input: {7, 7, 7, 7, 7}    Output: 7
Input: {-10, 2, 3, -4, 5}  Output: 5

=== Code Execution Successful ===
```

Complexity: Time: $O(n)$ - every element has to be checked once. Space: $O(1)$.

Question 6: Sort a List and Find the Maximum Element

Problem: Sort the list using an efficient sorting algorithm, then find the maximum element in the sorted list. Handle: empty list, single element list, and all elements equal.

Program (C):

```
#include <stdio.h>

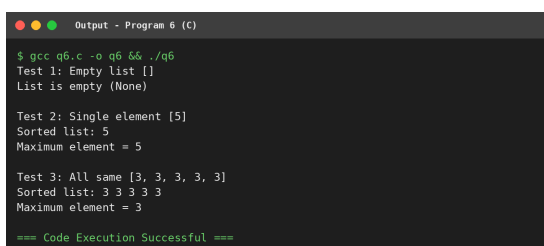
/* merge sort - O(n log n) */
void merge(int arr[], int l, int mid, int r) {
    int temp[100];
    int i = l, j = mid + 1, k = 0;
    while (i <= mid && j <= r) {
        if (arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }
    while (i <= mid)
        temp[k++] = arr[i++];
    while (j <= r)
        temp[k++] = arr[j++];
    for (i = l, k = 0; i <= r; i++, k++)
        arr[i] = temp[k];
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int mid = (l + r) / 2;
        mergeSort(arr, l, mid);
        mergeSort(arr, mid + 1, r);
        merge(arr, l, mid, r);
    }
}

void findMaxAfterSort(int arr[], int n) {
    int i;
    if (n == 0) {
        printf("List is empty (None)\n");
        return;
    }
    mergeSort(arr, 0, n - 1);
    printf("Sorted list: ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\nMaximum element = %d\n", arr[n - 1]); /* last element is the largest */
}

int main() {
    int t1[1];
    int t2[] = {5};
    int t3[] = {3, 3, 3, 3, 3};
    printf("Test 1: Empty list []\n");
    findMaxAfterSort(t1, 0);
    printf("\nTest 2: Single element [5]\n");
    findMaxAfterSort(t2, 1);
    printf("\nTest 3: All same [3, 3, 3, 3, 3]\n");
    findMaxAfterSort(t3, 5);
    return 0;
}
```

Output:



```
Output - Program 6 (C)
$ gcc q6.c -o q6 && ./q6
Test 1: Empty list []
List is empty (None)

Test 2: Single element [5]
Sorted list: 5
Maximum element = 5

Test 3: All same [3, 3, 3, 3, 3]
Sorted list: 3 3 3 3 3
Maximum element = 3

=== Code Execution Successful ===
```

Complexity: Time: $O(n \log n)$ for merge sort; maximum read in $O(1)$ from the end. Space: $O(n)$.

Question 7: Unique Elements from a List (Space Complexity)

Problem: Take a list of n numbers and create a new list containing only the unique elements. State the space complexity of the algorithm.

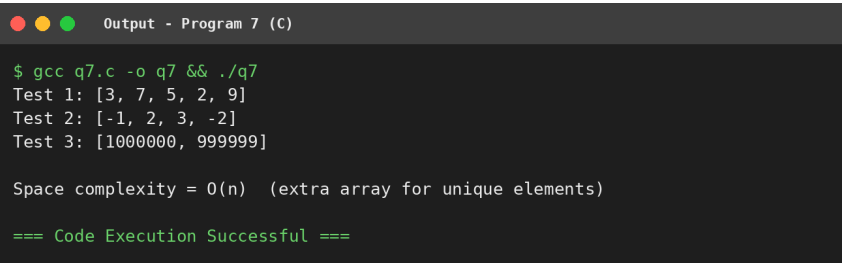
Program (C):

```
#include <stdio.h>

void printUnique(int arr[], int n) {
    int result[100];
    int count = 0, i, j;
    for (i = 0; i < n; i++) {
        int duplicate = 0;
        for (j = 0; j < count; j++) {
            if (result[j] == arr[i]) {
                duplicate = 1;
                break;
            }
        }
        if (duplicate == 0) {
            result[count] = arr[i];
            count++;
        }
    }
    printf("[");
    for (i = 0; i < count; i++) {
        printf("%d", result[i]);
        if (i != count - 1)
            printf(", ");
    }
    printf("]\n");
}

int main() {
    int a[] = {3, 7, 3, 5, 2, 5, 9, 2};
    int b[] = {-1, 2, -1, 3, 2, -2};
    int c[] = {1000000, 999999, 1000000};
    printf("Test 1: ");
    printUnique(a, 8);
    printf("Test 2: ");
    printUnique(b, 6);
    printf("Test 3: ");
    printUnique(c, 3);
    printf("\nSpace complexity = O(n) (extra array for unique elements)\n");
    return 0;
}
```

Output:



```
Output - Program 7 (C)

$ gcc q7.c -o q7 && ./q7
Test 1: [3, 7, 5, 2, 9]
Test 2: [-1, 2, 3, -2]
Test 3: [1000000, 999999]

Space complexity = O(n) (extra array for unique elements)

=== Code Execution Successful ===
```

Complexity: Time: $O(n^2)$ (each element is checked against the result list). Space complexity: $O(n)$ - the new list can hold up to n distinct elements.

Question 8: Bubble Sort with Big-O Analysis

Problem: Sort an array of integers using the bubble sort technique and analyse its time complexity using Big-O notation.

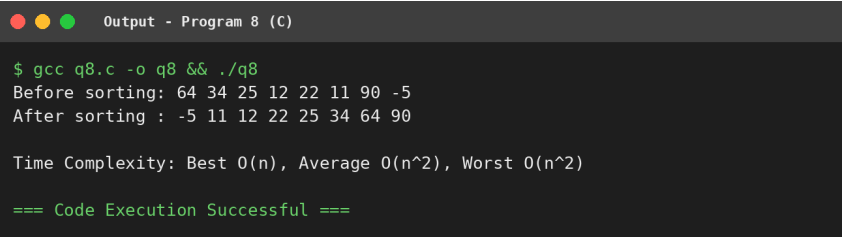
Program (C):

```
#include <stdio.h>

void bubbleSort(int arr[], int n) {
    int i, j, temp, swapped;
    for (i = 0; i < n - 1; i++) {
        swapped = 0;
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = 1;
            }
        }
        if (swapped == 0)
            break;
    }
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90, -5};
    int n = 8, i;
    printf("Before sorting: ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    bubbleSort(arr, n);
    printf("\nAfter sorting : ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n\nTime Complexity: Best O(n), Average O(n^2), Worst O(n^2)\n");
    return 0;
}
```

Output:



```
Output - Program 8 (C)

$ gcc q8.c -o q8 && ./q8
Before sorting: 64 34 25 12 22 11 90 -5
After sorting : -5 11 12 22 25 34 64 90

Time Complexity: Best O(n), Average O(n^2), Worst O(n^2)

=== Code Execution Successful ===
```

Complexity: Best case $O(n)$ (already sorted, thanks to the swapped flag), Average $O(n^2)$, Worst $O(n^2)$. Space: $O(1)$.

Question 9: Binary Search for Element x

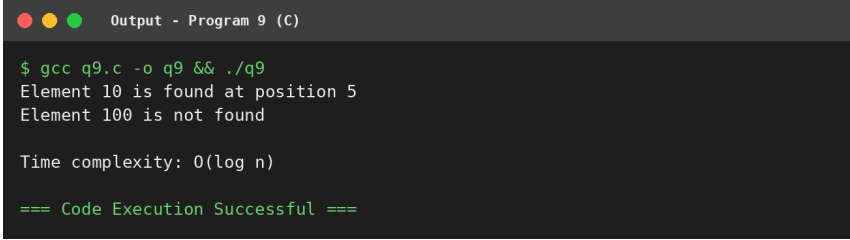
Problem: Check whether a given number x exists in an array using binary search and analyse its time complexity. Binary search needs sorted data and the sample array X is unsorted, so the program sorts the array first while remembering the original positions. The original 1-based position is printed, which matches the expected output (10 is at position 5).

Program (C):

```
#include <stdio.h>

int main() {
    int X[] = {3, 4, 6, -9, 10, 8, 9, 30};
    int n = 8, i, j, temp;
    int pos[8]; /* original positions of the elements */
    int keys[2] = {10, 100};
    int k;
    for (i = 0; i < n; i++)
        pos[i] = i + 1;
    /* binary search needs a sorted array, so sort first (keeping positions) */
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (X[j] > X[j + 1]) {
                temp = X[j]; X[j] = X[j + 1]; X[j + 1] = temp;
                temp = pos[j]; pos[j] = pos[j + 1]; pos[j + 1] = temp;
            }
        }
    }
    for (k = 0; k < 2; k++) {
        int low = 0, high = n - 1, found = -1;
        while (low <= high) {
            int mid = (low + high) / 2;
            if (X[mid] == keys[k]) {
                found = mid;
                break;
            }
            else if (X[mid] < keys[k])
                low = mid + 1;
            else
                high = mid - 1;
        }
        if (found != -1)
            printf("Element %d is found at position %d\n", keys[k], pos[found]);
        else
            printf("Element %d is not found\n", keys[k]);
    }
    printf("\nTime complexity: O(log n)\n");
    return 0;
}
```

Output:



```
$ gcc q9.c -o q9 && ./q9
Element 10 is found at position 5
Element 100 is not found

Time complexity: O(log n)

=== Code Execution Successful ===
```

Complexity: Binary search: Best $O(1)$, Average/Worst $O(\log n)$. Space: $O(n)$ for the array that stores the original positions.

Question 10: Sort an Array in $O(n \log n)$ with the Smallest Space Complexity

Problem: Given an array of integers `nums`, sort it in ascending order without using built-in functions, in $O(n \log n)$ time and with the smallest possible space complexity. Heap sort is used because it sorts in place.

Program (C):

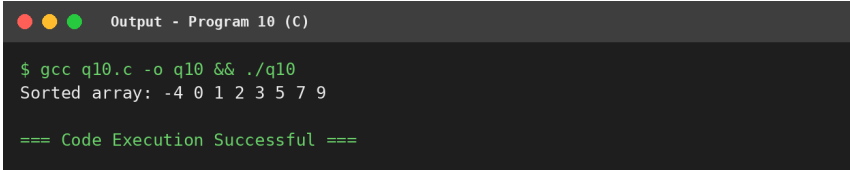
```
#include <stdio.h>

/* heap sort -  $O(n \log n)$  time and  $O(1)$  extra space */
void heapify(int nums[], int n, int i) {
    int largest, left, right, temp;
    while (1) {
        largest = i;
        left = 2 * i + 1;
        right = 2 * i + 2;
        if (left < n && nums[left] > nums[largest])
            largest = left;
        if (right < n && nums[right] > nums[largest])
            largest = right;
        if (largest == i)
            break;
        temp = nums[i];
        nums[i] = nums[largest];
        nums[largest] = temp;
        i = largest;
    }
}

void sortArray(int nums[], int n) {
    int i, temp;
    for (i = n / 2 - 1; i >= 0; i--)
        heapify(nums, n, i);
    for (i = n - 1; i > 0; i--) {
        temp = nums[0];
        nums[0] = nums[i];
        nums[i] = temp;
        heapify(nums, i, 0);
    }
}

int main() {
    int nums[] = {5, 2, 3, 1, 9, -4, 0, 7};
    int n = 8, i;
    sortArray(nums, n);
    printf("Sorted array: ");
    for (i = 0; i < n; i++)
        printf("%d ", nums[i]);
    printf("\n");
    return 0;
}
```

Output:



```
Output - Program 10 (C)

$ gcc q10.c -o q10 && ./q10
Sorted array: -4 0 1 2 3 5 7 9

=== Code Execution Successful ===
```

Complexity: Time: $O(n \log n)$ in best, average and worst cases. Space: $O(1)$ - in-place, iterative heapify.